

Build Anything, Show Your Thinking

GitHub Copilot practitioner · take-home assignment · ~4-6 hours · one repository + a 30-minute live defense

The task: build any small project you like. A CLI tool, an API, a scraper, a dashboard. We do not score the product. Keep it small, and spend your time on the five questions below.

Read this first. You can build this with Copilot, and you can write every document in the repo with Copilot. We assume you will. That means the artifacts themselves prove very little, so we score your **understanding and approach**: the decisions you made, the options you rejected, what went wrong, and whether you can defend it in conversation.

- **Evidence rule.** Every claim needs a receipt: a commit, a PR link, a session log, a real error message, the actual text of an approval prompt. Unsupported claims score zero for that item.
- **Don't squash your commits.** The history is part of the submission.
- Describing a config you couldn't run is fine. Claiming an outcome you never observed is not, and it does not survive the defense.
- If org policy blocks Copilot CLI, MCP, the cloud agent or plugin installs for you, say so and show what you would have configured. No penalty.

The five questions

Q1. How did you give Copilot its context, and what did you leave out?

Set up the files the agent reads on every task: a root `AGENTS.md` or `.github/copilot-instructions.md`, at least one path-scoped file in `.github/instructions/` using `applyTo`, and at least two prompt files in `.github/prompts/`.

- Show one instruction you added *because the agent got something wrong*, with the behaviour before and after.
- Tell us how the agent is supposed to prove its own work. Which command confirms a change is correct?
- Tell us what you deliberately left out. A long instruction file is a symptom, not an achievement.

Deliver: the files, plus a section in `WORKFLOW.md`.

Q2. Where did you hand work to an agent, and how did you know it was right?

Document **three** agentic sessions end to end, with evidence. Choose from: a task planned in plan mode where you edited the plan before any code; a multi-step session where the agent changed several files, ran tests, read the failures and corrected itself; a task delegated to the cloud agent that came back as a PR; a custom agent (`.github/agents/*.agent.md`) for a narrow recurring job; an autopilot-style run with an explicit stopping condition; or parallel sessions and how you kept them from colliding.

- State your rule for task sizing: what you handed over freely, what you kept on a short leash, and why.
- Include at least one failed iteration. "Tests passed" is the start of a verification story, not the end of one.
- Show one case where you fixed the *setup* (instructions, tools, prompt) rather than hand-patching the output.

Deliver: `WORKFLOW.md` with links to commits, PRs or transcripts.

Q3. What did you connect the agent to through MCP, and what does that cost you?

Configure at least one MCP server and commit the config. GitHub's server is fine, so is a filesystem, database, browser or docs server; one you wrote yourself scores well. Show one loop that could not have happened without it: the agent fetched external state, decided from it, acted, verified.

- Enable only the toolsets you need, and say why you scoped them that way.
- What happens when the server is slow, down, or returns something unexpected?
- If it returned attacker-controlled text, what in your setup limits the damage?
- What did those tools cost you in context, and what did you trim?

Deliver: the MCP config, plus your answers in `WORKFLOW.md`.

Q4. What can the agent do without asking you, and why is that the right line?

Plugins. Either install one from a marketplace and wire it in declaratively through `enabledPlugins` in `.github/copilot/settings.json` (with `extraKnownMarketplaces` if needed), or author a small one: `plugin.json` plus a skill, agent, hook or MCP server. Explain why a plugin rather than repo config, and **how you vetted it before enabling it**. A plugin ships agents, skills, hooks and MCP servers that run with your local permissions.

Approvals. Write `APPROVALS.md` covering: standing allows and the reason for each; hard denies (deny beats allow, including allow-all); whether you narrowed the tool surface the model can even see; what persisted to `permissions-config.json` or `allowedUrls` and whether you reset it; 3-5 real approval prompts with what you chose and why, **at least one a refusal**; any sandboxing; and which choices would break under an enterprise policy that disables MCP or blocks allow-all.

Marked down: running under `--allow-all` / `--yolo` and calling it a productivity win. Marked up: a posture that got tighter as you learned what the agent actually needed, with the diff to prove it.

Deliver: plugin config or source, and `APPROVALS.md`.

Q5. What were the real forks in the road, and what didn't work?

Write `DECISIONS.md`: six to ten terse entries, one per genuine decision. Each needs the choice, the alternatives you considered, the cost you accepted, **and what would change your mind**. An entry missing the road not taken is a description, not a decision.

Then close `WORKFLOW.md` with a required section titled **"What didn't work"**: dead ends, misfires, features that fought you, things you abandoned. A submission with nothing here reads as either dishonest or unreflective. Add one thing you would do differently with another week, and a short note on how you used Copilot to produce this submission.

Deliver: `DECISIONS.md` and the closing sections of `WORKFLOW.md`.

Scoring

Written submission — 60 points	Live defense — 40 points
Q5 decision log and honesty — 15 Q2 agentic workflow, with evidence — 15 Q4 approvals and permission governance — 10 Q3 MCP integration — 8 Q1 context and instruction layer — 7 Q4 plugins — 5	Explains the concepts in your own words — 15 Defends your choices under questioning — 15 Adapts on the spot to a changed constraint — 10

The live defense (30 minutes)

Not a surprise exam. Preparing for it is the skill, so here is the shape.

- **Explain in plain language:** what actually changes between plan, agent and autopilot; what MCP is and is not, and where it sits next to instructions, prompt files, agents, skills and plugins; what a plugin can do to your machine once enabled; why deny rules beat allow rules and where approvals persist between sessions.
- **Defend:** one permission you granted and one you refused; something in your instruction file you would delete if we made you halve it; the moment you stopped trusting the agent and what triggered it.
- **Redesign live (~10 min):** we change one constraint. MCP is banned, or your team just doubled, or the agent may never touch a file outside one directory. You rework your setup out loud. We watch the reasoning, not the landing.

If you cannot defend part of your submission, say so. "Copilot drafted that and I didn't verify it" scores better than being caught.

Submission

Share the repository link (add the reviewers if private), unsquashed history, with `WORKFLOW.md`, `DECISIONS.md` and `APPROVALS.md` at the root and a short `README.md` saying what it is, how to run it and how long you spent. Link transcripts and screenshots; don't paste walls of them.

Not scored: how impressive the project is, lines of code, how much Copilot wrote, polished prose, or using every feature listed. "I skipped X because Y" is worth more than a hollow demo of X.